Практическая работа: тестирование AuthManager

Цель: проверить регистрацию и аутентификацию пользователей, подсчёт по странам, удаление пользователей и перевод средств с помощью Pytest.

Состав выполненных проверок

Требование задания	Реализация
3 базовых теста	Регистрация, успешная аутентификация, удаление пользователя
3 параметризованных случая	Подсчёт пользователей для CountryA, CountryB и CountryC
2 теста исключений	Недостаточно средств; недопустимая сумма перевода
Фикстура SQLite	Отдельная база данных :memory: для каждого теста
Метки	basic, exception, security
SQL-инъекция	Проверка, что строка атаки не проходит аутентификацию
Пропускаемый тест	pytest.mark.skip

Важное замечание по безопасности

В исходном варианте задания SQL-запросы формируются через format(), что создаёт уязвимость SQL-инъекции. В подготовленной версии использованы placeholders ? и отдельная передача параметров. Поэтому тест проверяет безопасное поведение исправленного класса, а не подтверждает безопасность исходного уязвимого кода.

1. Код класса auth_manager.py

```
import sqlite3


class AuthManager:
    """Управление пользователями и их балансами в SQLite."""

    def __init__(self, connection):
        self.connection = connection
        self.create_tables()

    def create_tables(self):
        with self.connection:
            self.connection.execute("""
                CREATE TABLE IF NOT EXISTS users (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    username TEXT NOT NULL UNIQUE,
                    password TEXT NOT NULL,
                    country TEXT NOT NULL,
                    balance REAL NOT NULL
                )
            """)

    def register_user(self, username, password, country, balance):
        # Параметры SQL передаются отдельно от текста запроса.
        with self.connection:
            cursor = self.connection.execute("""
                INSERT INTO users (username, password, country, balance)
                VALUES (?, ?, ?, ?)
            """, (username, password, country, balance))
        return cursor.lastrowid

    def authenticate_user(self, username, password):
        cursor = self.connection.execute("""
            SELECT * FROM users WHERE username = ? AND password = ?
        """, (username, password))
        return cursor.fetchone()

    def delete_user(self, user_id):
        with self.connection:
            self.connection.execute("DELETE FROM users WHERE id = ?", (user_id,))

    def get_user_by_id(self, user_id):
        cursor = self.connection.execute(
            "SELECT * FROM users WHERE id = ?", (user_id,)
        )
        return cursor.fetchone()

    def count_users_by_country(self, country):
        cursor = self.connection.execute(
            "SELECT COUNT(*) FROM users WHERE country = ?", (country,)
        )
        return cursor.fetchone()[0]

    def transfer_balance(self, from_user_id, to_user_id, amount):
        if amount <= 0:
            raise ValueError("Transfer amount must be positive")

        with self.connection:
            cursor = self.connection.execute(
                "SELECT balance FROM users WHERE id = ?", (from_user_id,)
            )
```

```
row = cursor.fetchone()
if row is None:
    raise ValueError("Sender not found")
if row[0] < amount:
    raise ValueError("Insufficient funds")

recipient = self.connection.execute(
    "SELECT id FROM users WHERE id = ?", (to_user_id,)
).fetchone()
if recipient is None:
    raise ValueError("Recipient not found")

self.connection.execute(
    "UPDATE users SET balance = balance - ? WHERE id = ?",
    (amount, from_user_id)
)
self.connection.execute(
    "UPDATE users SET balance = balance + ? WHERE id = ?",
    (amount, to_user_id)
)
```

2. Код автоматических тестов test_auth_manager.py

```
import sqlite3
import pytest

from auth_manager import AuthManager


@pytest.fixture
def auth_manager():
    """Для каждого теста создаётся отдельная база SQLite в памяти."""
    connection = sqlite3.connect(":memory:")
    manager = AuthManager(connection)
    yield manager
    connection.close()


# БАЗОВЫЕ ТЕСТЫ (3)
@pytest.mark.basic
def test_register_user(auth_manager):
    user_id = auth_manager.register_user("anna", "pass123", "Germany", 500)
    user = auth_manager.get_user_by_id(user_id)
    assert user is not None
    assert user[1] == "anna"


@pytest.mark.basic
def test_authenticate_user_success(auth_manager):
    auth_manager.register_user("ivan", "secret", "Poland", 300)
    assert auth_manager.authenticate_user("ivan", "secret") is not None


@pytest.mark.basic
def test_delete_user(auth_manager):
    user_id = auth_manager.register_user("olga", "pw", "France", 100)
    auth_manager.delete_user(user_id)
    assert auth_manager.get_user_by_id(user_id) is None


# ПАРАМЕТРИЗОВАННЫЙ ТЕСТ: 3 набора данных
@pytest.mark.parametrize(
    "country, expected_count",
    [
        ("CountryA", 2),
        ("CountryB", 1),
        ("CountryC", 0),
    ],
)
def test_count_users_by_country(auth_manager, country, expected_count):
    auth_manager.register_user("user1", "password123", "CountryA", 1000)
    auth_manager.register_user("user2", "password123", "CountryA", 1000)
    auth_manager.register_user("user3", "password123", "CountryB", 1000)
    assert auth_manager.count_users_by_country(country) == expected_count


# ТЕСТЫ ИСКЛЮЧЕНИЙ (2)
@pytest.mark.exception
def test_transfer_insufficient_funds(auth_manager):
    sender = auth_manager.register_user("sender", "pw", "Germany", 20)
    receiver = auth_manager.register_user("receiver", "pw", "Germany", 10)
    with pytest.raises(ValueError, match="Insufficient funds"):
        auth_manager.transfer_balance(sender, receiver, 50)


@pytest.mark.exception
def test_transfer_non_positive_amount(auth_manager):
    sender = auth_manager.register_user("sender", "pw", "Germany", 100)
    receiver = auth_manager.register_user("receiver", "pw", "Germany", 0)
    with pytest.raises(ValueError, match="positive"):
        auth_manager.transfer_balance(sender, receiver, 0)

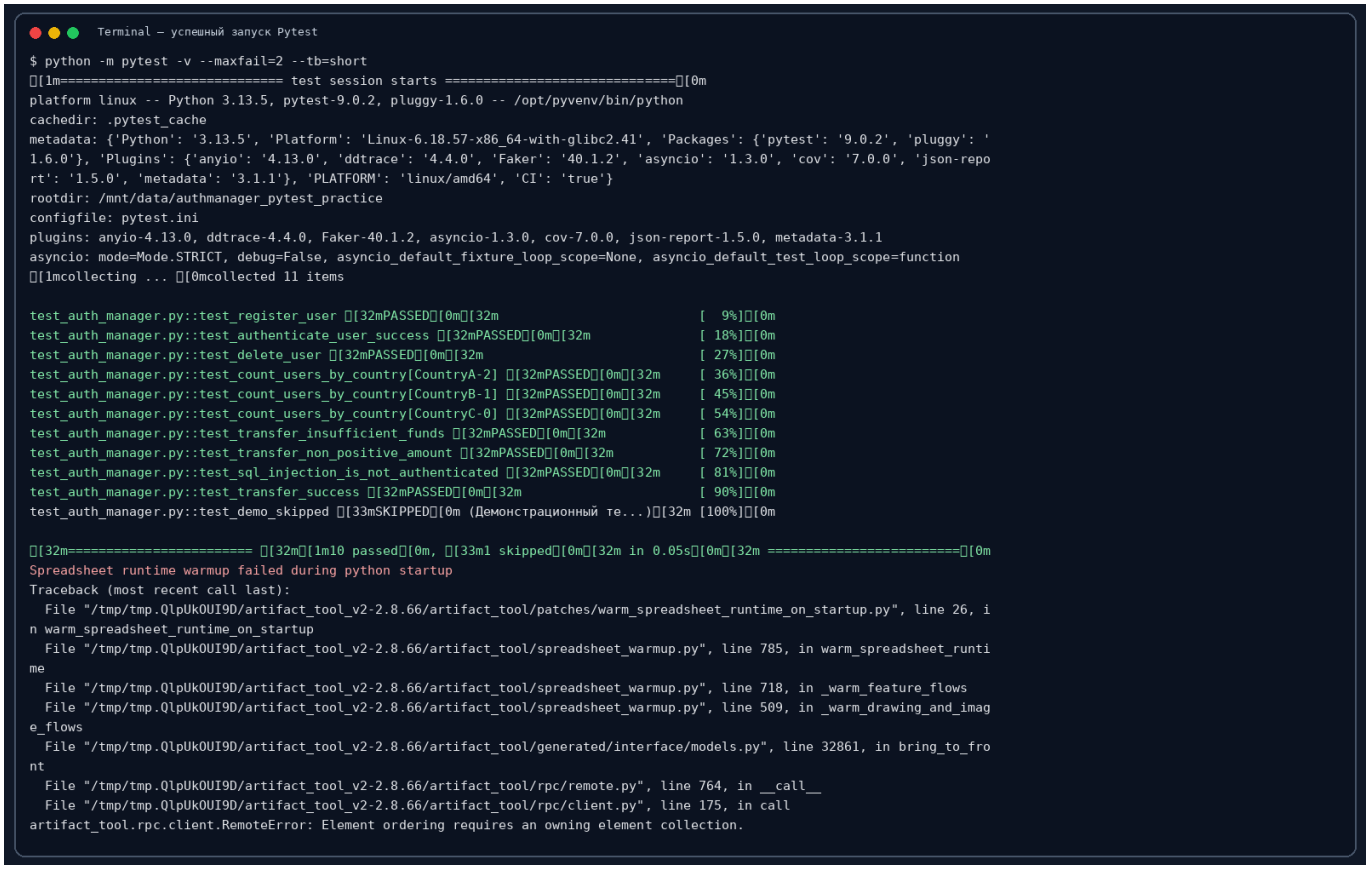

# ОБЯЗАТЕЛЬНЫЙ ТЕСТ НА SQL-ИНЪЕКЦИЮ
@pytest.mark.security
def test_sql_injection_is_not_authenticated(auth_manager):
    auth_manager.register_user("ordinary_user", "safe_password", "Germany", 100)
    attack = "' OR '1'='1' --"
    result = auth_manager.authenticate_user(attack, "anything")
    assert result is None


# ДОПОЛНИТЕЛЬНЫЙ ТЕСТ ПЕРЕВОДА СРЕДСТВ
@pytest.mark.basic
def test_transfer_success(auth_manager):
    sender = auth_manager.register_user("alice", "pw", "Germany", 100)
    receiver = auth_manager.register_user("bob", "pw", "Germany", 25)
    auth_manager.transfer_balance(sender, receiver, 30)
    assert auth_manager.get_user_by_id(sender)[4] == 70
    assert auth_manager.get_user_by_id(receiver)[4] == 55


# ПРОПУСКАЕМЫЙ ТЕСТ — демонстрация pytest.mark.skip
@pytest.mark.skip(reason="Демонстрационный тест намеренно пропущен")
def test_demo_skipped():
    assert 1 == 1
```

3. Успешный запуск тестов

Команда: python -m pytest -v --maxfail=2 --tb=short. Снимок фактического вывода:



4. Неуспешный запуск — пример обнаруженной ошибки

Для демонстрации отрицательного результата отдельно запущен намеренно ошибочный тест: ожидалось 5, фактически получено 4. Он не входит в основной набор AuthManager.



5. Запуск на компьютере

```
python -m pip install -r requirements.txt
python -m pytest -v --maxfail=2 --tb=short
python -m pytest -v -m security
python -m pytest -v -m exception
```

Вывод: основной набор завершился с результатом 10 passed, 1 skipped. Отдельный демонстрационный тест завершился ошибкой 1 failed.